ЛАБОРАТОРНАЯ РАБОТА №2 • ДИСЦИПЛИНА: ИНФОРМАТИКА (3 СЕМЕСТР)

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
In-Memory Кеширующий Модуль с Алгоритмом Вытеснения LRU

1. НАЗНАЧЕНИЕ И ЦЕЛИ СИСТЕМЫ

Настоящее техническое задание определяет требования к подсистеме промежуточного хранения данных (In-Memory LRU Cache). Главной целью системы является ускорение повторного доступа к объектам за счет хранения наиболее востребованных записей в оперативной памяти и автоматического ограничения объёма используемой памяти по алгоритму **Least Recently Used (LRU)**.

2. АБСТРАКТНЫЕ ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

Описание логики поведения подсистемы сгруппировано на абстрактном уровне и не зависит от конкретного языка программирования или именования функций.

ФТ-1: Абстрактный контракт словаря (Ассоциативная память)

Система должна предоставлять единый абстрактный интерфейс доступа к данным по принципу «Ключ — Значение».

- Ключи в системе должны быть уникальными. Попытка записи по существующему ключу обновляет ассоциированное значение.
- Подсистема должна поддерживать базовые операции: сохранения, чтения по ключу, явного удаление, полного сброса состояния и получения количества текущих элементов.

ФТ-2: Логика управления свежестью (LRU Policy)

- Каждая операция обращения к данным (сохранение или успешное чтение по ключу) делает соответствующую запись «наиболее недавно использованной» (MRU).
- Система обязана постоянно поддерживать абсолютный хронологический порядок актуальности всех хранящихся элементов.

ФТ-3: Автоматическое вытеснение при переполнении (LRU Eviction)

Максимальная вместимость кеша строго ограничена фиксированным предельным значением N (емкостью).

- При выполнении операции записи в момент, когда количество элементов достигло предельной емкости N, система обязана автоматически вытеснить из памяти «наименее недавно использовавшийся элемент» (LRU).
- Вытесненный элемент полностью удаляется из кеша, высвобождая место под новый элемент.

Концептуальный принцип вытеснения: Выбор удаляемого элемента при переполнении определяется исключительно историей хронологического доступа к нему, а не значением его ключа или содержимым.

3. ОТОБРАЖЕНИЕ АБСТРАКТНЫХ ТРЕБОВАНИЙ НА АРХИТЕКТУРУ

Для обеспечения максимальной производительности (O(1)) подсистема реализуется в виде гибридного модуля, объединяющего два специализированных компонента.

Абстрактное требование	Архитектурный компонент	Назначение компонента
Индексация и быстрый поиск	Хеш-таблица (`IDictionary`)	Является основным владельцем данных (пары Key-Value). Организует прямой поиск по ключу за константное время O(1) с использованием хеш-функции, цепочек коллизий и динамического рехеширования.
Хронология использования (LRU)	Двусвязный список	Служит хронологической цепочкой актуальности. Позволяет за константное время O(1) переставлять элементы в начало (MRU) и удалять из хвоста (LRU).
Единый интерфейс доступа	Абстрактный интерфейс словаря (`IDictionary`)	Инкапсулирует внутреннюю гибридную логику, предоставляя наружному клиенту стандартные методы работы с ассоциативной памятью.

4. ГАРАНТИИ ВРЕМЕННОЙ СЛОЖНОСТИ (BIG-O)

Операция подсистемы	Гарантированная сложность	Пояснение логики выполнения
Поиск элемента по ключу	O(1)	Прямое обращение по хеш-индексу в словаре за O(1) и перенос элемента в начало хронологической цепочки за O(1).
Вставка / Обновление записи	O(1)	Добавление пары в хеш-таблицу за O(1) и размещение в голове хронологического списка за O(1).
Вытеснение LRU-элемента	O(1)	Получение наименее актуального элемента из хвоста хронологической цепочки и его удаление из словаря за O(1).
Обновление приоритета свежести	O(1)	Перемещение элемента в начало хронологической цепочки при любом обращении выполняется за константное время O(1).